

Personalized Learning Path Generator

Full technical documentation — architecture, the adaptive engine, the ML models, API reference, and database setup.

Contents

1. Overview & Quickstart
2. Architecture
3. The Adaptive Engine
4. ML Models
5. API Reference
6. Database

Personalized Learning Path Generator

An adaptive Class 8 Mathematics practice tool. It diagnoses a student's level, adapts question difficulty in real time using a trained ML model, tracks per-topic mastery with an Elo-style algorithm, and gives teachers a live dashboard — a heatmap, ML-derived learning archetypes, and per-student drill-down.

Built as a hackathon prototype: one full-stack Next.js app (no separate backend service), SQLite-simple-to-Postgres database via Prisma, and a small set of genuinely trained (not hand-coded) ML models whose weights are exported as plain JSON and read at runtime with zero Python dependency.

Full documentation lives in [docs/](#):

Doc	Covers
docs/ARCHITECTURE.md	How the pieces fit together, folder structure, request flow
docs/ADAPTIVE-ENGINE.md	The mastery-tracking + question-selection algorithm, line by line
docs/ML-MODELS.md	All 5 trained models: what they predict, training data, accuracy, how to retrain
docs/API.md	Every API route: method, params, request/response shape
docs/DATABASE.md	Prisma schema, Postgres setup, migrations

Quickstart

```
npm install
npx prisma migrate dev # applies schema to Postgres
npm run seed           # loads 4 topics x 10 questions
npm run dev            # http://localhost:3000
```

Requires a running Postgres instance and `DATABASE_URL` set in `.env` (see [docs/DATABASE.md](#) for local setup). No API keys required — there is no external LLM call in this build.

To populate the teacher dashboard with realistic-looking data before a demo:

```
BASE_URL=http://localhost:3000 npx tsx scripts/simulate-demo-data.ts
```

To retrain the ML models (optional — trained weights are already committed to `lib/models/*.json`):

```
python3 -m venv venv && source venv/bin/activate
pip install scikit-learn numpy
python scripts/train_difficulty_model.py
python scripts/train_cluster_model.py
python scripts/train_time_to_mastery_model.py
python scripts/train_topic_priority_model.py
python scripts/train_at_risk_model.py
```

Tech stack

- **Next.js 16** (App Router) + TypeScript
- **Tailwind CSS 4**
- **Prisma 6** + **PostgreSQL** (local, via Homebrew)
- **Recharts** (installed; the dashboard heatmap is hand-rolled HTML/CSS per accessibility guidance — see [docs/ARCHITECTURE.md](#))
- **scikit-learn** (Python, training-time only) — logistic/linear regression and KMeans, exported to JSON for zero-dependency TypeScript inference

What's genuinely "ML" here vs. hand-coded

Two things are commonly hand-coded in prototypes like this and were instead **trained on data** (synthetic, since there's no real student data yet):

1. **Question difficulty selection** — a logistic regression, not an if/else ladder. See [docs/ML-MODELS.md](#).
2. **Student learning archetypes** on the teacher dashboard — a KMeans clustering model, not a hardcoded rule. See [docs/ML-MODELS.md](#).

Three more models were built alongside these (time-to-mastery estimation, class-wide topic-priority ranking, an at-risk/declining-trend detector) — trained and tested, but not all are wired into the UI. See [docs/ML-MODELS.md](#) for the full picture, including honest accuracy numbers and where each model's fit is genuinely strong vs. weak.

Architecture

Why one Next.js app, not separate services

The scope decision from day one: build this as **one full-stack Next.js (App Router) application** in TypeScript, not separate frontend/backend services. The "adaptive engine" and "ML inference" logic live in plain TypeScript modules under `lib/`, called directly from Next.js API routes. This keeps the whole thing runnable with one `npm run dev` and trivially deployable, while still being accurate to describe as "an adaptive engine with a trained ML model" — it genuinely is one, just not a microservice.

Folder structure

app/	
page.tsx	Landing page (role picker: Student / Teacher)
quiz/page.tsx	Student quiz UI (diagnostic + adaptive)
revision/page.tsx	Student's weak-topic list + "Practice this"
dashboard/page.tsx	Teacher heatmap + student drill-down
api/	
student/route.ts	POST create student, GET list students
student/[id]/route.ts	GET one student's masteries/weak topics/attempts
quiz/next/route.ts	GET next question (diagnostic or adaptive)
quiz/answer/route.ts	POST an answer, updates Attempt + Mastery
dashboard/route.ts	GET all students shaped for the heatmap
lib/	
prisma.ts	Prisma client singleton
session.ts	Client-side localStorage session (no auth)
colors.ts	Mastery color-band mapping (status palette)
adaptive-engine.ts	Core algorithm: pure functions, no DB access
quiz-service.ts	DB-aware layer: wraps adaptive-engine with
	Prisma queries, handles the diagnostic phase
ml/	
difficulty-model.ts	Inference for Model 1 (wired into engine)
cluster-model.ts	Inference for Model 2 (wired into dashboard)
time-to-mastery-model.ts	Inference for Model 3 (trained, not wired)
topic-priority-model.ts	Inference for Model 4 (trained, not wired)
at-risk-model.ts	Inference for Model 5 (trained, not wired)
models/	
*.json	Trained weights, one file per model
prisma/	
schema.prisma	Data model (Student, Topic, Question, Attempt, Mastery)
seed.ts	Loads the 4 topics x 10 questions
migrations/	SQL migration history
scripts/	
train*.py	Python: generates synthetic data, trains a
	model, writes lib/models/*.json + data/*.csv
test*.ts	TypeScript: sanity-checks each ml/*.ts layer
test-engine.ts	Simulates 30 attempts through the adaptive
	engine, prints the mastery trajectory
simulate-demo-data.ts	Plays 3 fake students through the real
	engine (via Prisma directly) to seed a
	realistic-looking teacher dashboard
data/	
*.csv	The synthetic training data each Python
	script generates (200k rows each)

```
components/
  MasteryBar.tsx           Animated per-topic mastery bar (quiz page)
```

Request flow: a student answers a question

```
Browser (app/quiz/page.tsx)
-> GET /api/quiz/next?studentId=...
  app/api/quiz/next/route.ts
  -> lib/quiz-service.ts: getNextQuestion()
    - counts the student's total attempts
    - if < 8: diagnostic phase (see ADAPTIVE-ENGINE.md)
    - else: adaptive phase
      -> lib/adaptive-engine.ts: selectTargetTopic()
        -> lib/ml/difficulty-model.ts: selectDifficulty()
          (Model 1 – logistic regression inference)
      -> lib/adaptive-engine.ts: selectQuestion()
    <- { question, reason, phase }
-> student picks an option
-> POST /api/quiz/answer { studentId, questionId, selectedIdx }
  app/api/quiz/answer/route.ts
  - looks up the question's real answerIdx server-side (never sent
    to the client) to determine correct/incorrect
  - lib/adaptive-engine.ts: updateMastery() – Elo-style update
  - upserts the Mastery row in Postgres
  <- { correct, correctIdx, mastery }
-> mastery bar animates, feedback + reason shown
```

Request flow: teacher dashboard

```
Browser (app/dashboard/page.tsx)
-> GET /api/dashboard
  app/api/dashboard/route.ts
  - fetches all students with their Mastery rows
  - builds a 4-number mastery vector per student, in the FIXED
    topic order the cluster model was trained on (Fractions,
    Ratios, Linear Equations, Percentages) – order matters here,
    since it must match training time exactly
  -> lib/ml/cluster-model.ts: assignArchetype()
    (Model 2 – nearest-centroid KMeans inference)
  - computes weak topics per student (lib/adaptive-engine.ts)
  <- { topics, students: [{ masteryByTopic, archetype, weakTopics, ... }] }
-> heatmap renders (color-coded cells, archetype badge per student)
-> clicking a student -> GET /api/student/[id] for attempt history
```

No authentication, by design

This is a demo prototype: the landing page is a hardcoded role picker (Student / Teacher), no passwords. A student's identity is just a `cuid()` stored in `localStorage` (`lib/session.ts`) — there's no session/JWT/cookie layer. Do not use this auth pattern outside a demo context.

Database: SQLite -> Postgres

The build started on SQLite (Prisma's zero-setup option for a fast start), then was migrated to a local PostgreSQL instance (via Homebrew) so the data could be inspected in DBeaver like a normal relational database. See [DATABASE.md](#) for the full schema and connection details. The switch only touched `prisma/schema.prisma`'s `datasource` block and `DATABASE_URL` — application code is unaware of which database engine is behind Prisma.

Why the heatmap isn't a Recharts component

Recharts is installed (per the original tech-stack spec) but has no native heatmap primitive. The teacher dashboard's topic x student grid is hand-rolled as a colored HTML table instead, following a dataviz accessibility pass: cells use a fixed 3-step status palette (red/amber/green) with the percentage always printed as visible text inside the cell — so identity/severity is never color-alone, and the page works for colorblind viewers without relying on hue discrimination.

The Adaptive Engine

Source: [lib/adaptive-engine.ts](#) (pure functions, no database access — fully unit-testable) and [lib/quiz-service.ts](#) (the DB-aware layer that calls it).

1. Mastery tracking (Elo-inspired)

Every student has a `Mastery` row per topic: a float in `[0.05, 0.95]` starting at `0.5` (assumed 50/50 before any evidence). After every answered question, `updateMastery()` nudges it toward the observed outcome:

```
// lib/adaptive-engine.ts
const LEARNING_RATE = 0.15; // "K" — how fast mastery reacts to new evidence

function updateMastery(currentMastery: number, correct: boolean): number {
  const expected = currentMastery; // treat current score as P(correct)
  const actual = correct ? 1 : 0;
  const next = currentMastery + LEARNING_RATE * (actual - expected);
  return clamp(next, 0.05, 0.95); // never fully certain either way
}
```

This is the same shape as an Elo rating update: the score moves toward whatever just happened, by an amount proportional to how surprising the outcome was (a correct answer from someone with `mastery=0.9` barely moves the needle; a correct answer from someone at `mastery=0.1` moves it a lot).

The clamp to `[0.05, 0.95]` matters for two reasons: it keeps the score usable as a probability-like input to the ML difficulty model (which can't handle exactly 0 or 1), and it means no single answer can ever fully "prove" mastery or its absence — consistent with treating this as a noisy signal, not ground truth.

2. Diagnostic phase (first 8 questions)

Before there's any evidence, the app can't yet tell which topic is weak. So the first `topics.length × 2 = 8` questions cycle through each of the 4 topics twice, always at difficulty 2 (the "medium" band), to seed a rough mastery baseline for every topic before switching to adaptive mode. This logic lives in `getNextQuestion()` in `lib/quiz-service.ts`:

```
const topicOrder = [Fractions, Ratios, LinearEq, Percentages, // round 1
                    Fractions, Ratios, LinearEq, Percentages]; // round 2
const targetTopicId = topicOrder[totalAttemptsSoFar];
```

It picks a difficulty-2 question in that topic the student hasn't already been asked, falling back to "any difficulty-2 question in that topic" if they've somehow exhausted the unseen ones.

3. Adaptive phase (after the diagnostic)

Once 8+ attempts exist, every subsequent question goes through `selectTargetTopic()` then `selectQuestion()`.

Picking the topic: weakest-first

```
function selectTargetTopic(masteries, restrictToTopicId?) {
  const candidates = restrictToTopicId
    ? masteries.filter(m => m.topicId === restrictToTopicId) // "Practice this" mode
    : masteries.filter(m => m.score < MASTERY_THRESHOLD); // 0.8 – not yet mastered
  const pool = candidates.length > 0 ? candidates : masteries; // fallback: everything mastered
  const weakest = [...pool].sort((a, b) => a.score - b.score)[0];
  ...
}
```

Among topics not yet "mastered" (`score < 0.8`), it picks the single weakest one and serves a question there. This is a greedy strategy — it will keep hammering the same weak topic across many consecutive questions until that topic's mastery rises above whatever the next-weakest topic is. That's intentional: the goal is remediation, not balanced topic rotation.

If `restrictToTopicId` is set (the `/revision` page's "Practice this" button, via `?topic=<id>` on the quiz URL), the topic choice is skipped entirely and every question comes from that one topic.

Picking the difficulty: the ML model, not a hand-coded ladder

```
const targetDifficulty = selectDifficulty(weakest.score); // lib/ml/difficulty-model.ts
```

This calls **Model 1** (logistic regression — see [ML-MODELS.md](#)), which was trained to predict `P(correct | mastery, difficulty)` and picks whichever of difficulty 1/2/3 puts the student closest to a 75% success rate — the "desirable difficulty" zone from learning science. This replaced an earlier hand-coded `if (score < 0.4) return 1; ... ladder`.

The explainability string

Every returned question comes with a plain-English `reason` :

```
"Recommended because your Ratios mastery is 22% – lower than your other topics (avg 61%)."
```

or, in "Practice this" mode:

```
"Practicing Ratios – your mastery here is 22%."
```

This is a template string (not an LLM call — an earlier plan considered calling Claude per-question for this, but that was never wired in; see the "What's genuinely ML" note in the root README). It's shown on the student quiz UI as a named feature so the reasoning behind each question is never opaque.

Picking the specific question: no-repeat rule

```
function selectQuestion(questions, targetTopicId, targetDifficulty, recentAttempts) {
  const recentOnTopic = recentAttempts.filter(a => a.topicId === targetTopicId).slice(-5);
  const recentlyCorrectIds = new Set(recentOnTopic.filter(a => a.correct).map(a => a.questionId));
  const eligible = topicQuestions.filter(q =>
    q.difficulty === targetDifficulty && !recentlyCorrectIds.has(q.id)
  );
  ...
}
```

Within the target topic and difficulty, a question already answered *correctly* in the last 5 attempts on that topic is excluded (so a student doesn't just see the same easy win repeatedly). If that leaves nothing, it progressively relaxes: same difficulty regardless of recency, then any difficulty not recently seen, then anything at all — the app should never hard-stop just because the no-repeat rule ran out of room in a 10-question topic pool.

4. Weak-topic / revision plan

```
const WEAK_THRESHOLD = 0.5;
const WEAK_MIN_ATTEMPTS = 3;

function getWeakTopics(masteries, attemptCounts) {
  return masteries
    .filter(m => m.score < WEAK_THRESHOLD && attemptCounts[m.topicId] >= WEAK_MIN_ATTEMPTS)
    .sort((a, b) => a.score - b.score);
}
```

A topic only counts as "weak" (and shows up on `/revision`) once there's been at least 3 attempts on it — otherwise a single unlucky first answer would immediately flag a topic the student hasn't really been tested on yet. Sorted weakest-first so the most urgent topic is at the top.

Testing this in isolation

`scripts/test-engine.ts` simulates 30 attempts for a fake student with a known skill profile per topic (e.g. 80% success on Fractions, 20% on Ratios) and prints the mastery trajectory round by round — useful for eyeballing that the engine converges sensibly without needing the database or the UI at all:

```
npx tsx scripts/test-engine.ts
```

ML Models

Five models were trained. All follow the same pattern: a Python script generates synthetic data (there's no real student data yet — this is a fresh prototype), trains a small scikit-learn model, evaluates it on a held-out 20% test split, and exports the learned weights as JSON. A TypeScript module then re-implements the model's math by hand (a sigmoid, a dot product, a nearest-centroid search) so the Next.js app can run inference with **zero Python dependency at runtime** — Python is only needed to retrain.

#	Model	Type	Wired into the app?
1	Difficulty selection	Logistic regression	[Yes] <code>lib/adaptive-engine.ts</code>
2	Archetype clustering	KMeans	[Yes] <code>/dashboard</code>
3	Time-to-mastery	Linear regression (quadratic features)	[No] trained + tested only
4	Topic priority	Linear regression	[No] trained + tested only
5	At-risk detector	Logistic regression	[No] trained + tested only

All five report **held-out accuracy in the 85-90% range** (see each section for exact numbers). Where the ground-truth generating formula is known, the script also computes the **Bayes-optimal ceiling** — the best any model could possibly score given the label noise baked into the data — so the reported accuracy is interpretable rather than a bare number.

Model 1 — Difficulty selection {#model-1}

File: `scripts/train_difficulty_model.py -> lib/models/difficulty-model.json -> lib/ml/difficulty-model.ts`

Predicts: `P(correct | mastery, difficulty)` — used to pick whichever difficulty (1/2/3) puts a student closest to a 75% success rate (the "desirable difficulty" zone).

Training data (`data/difficulty_training_data.csv` , 200,000 rows, columns `mastery`, `difficulty`, `correct`): for each row, `mastery` is uniform on [0,1], `difficulty` is uniform on {1,2,3}, and the ground truth is `p = sigmoid(11 * (mastery - 0.2 * difficulty))`, with `correct` sampled as a coin flip from that probability. The `11` steepness constant controls how much overlap/noise exists between classes — it's the main knob used to land held-out accuracy in the 85-90% band.

Held-out results (20% test split, 40,000 rows):

Metric	Value
Test accuracy	87.8%
Test ROC AUC	0.95

Metric	Value
Bayes-optimal ceiling	87.8%

The model's accuracy essentially equals the Bayes ceiling — it has fully recovered the true relationship; the remaining "error" is pure label noise that no model could avoid.

Learned weights: mastery: +11.0, difficulty: -2.2 — signs asserted in the training script (mastery coefficient must be positive, difficulty must be negative) before the model is allowed to export.

TypeScript inference:

```
// lib/ml/difficulty-model.ts
export function predictCorrectProb(mastery: number, difficulty: number): number {
  const z = weights.mastery * mastery + weights.difficulty * difficulty + bias;
  return 1 / (1 + Math.exp(-z));
}

export function selectDifficulty(mastery: number): 1 | 2 | 3 {
  // returns whichever of {1,2,3} gives predictCorrectProb closest to 0.75
}
```

Model 2 — Student archetype clustering {#model-2}

File: `scripts/train_cluster_model.py` -> `lib/models/cluster-centroids.json` -> `lib/ml/cluster-model.ts`

Predicts: which of 3 learning archetypes a student's 4-topic mastery vector belongs to — "On Track", "Needs Support", or "Uneven — targeted help needed" — shown as a color-coded badge next to each student on the teacher dashboard.

Training data (`data/cluster_training_data.csv`, ~200,000 rows, columns `pattern`, `fractions`, `ratios`, `linear_equations`, `percentages`): three generating patterns, ~66,666 rows each —

- **uniformly strong:** all 4 topics ~0.8 + Gaussian noise
- **uniformly weak:** all 4 topics ~0.25 + Gaussian noise
- **spiky:** Fractions & Linear Equations ~0.8, Ratios & Percentages ~0.25
- Gaussian noise (a *fixed* high/low split — see the bug note below)

Noise std dev is `0.30`, tuned so the archetypes overlap enough to land held-out accuracy in the 85-90% range instead of a suspiciously perfect 100%.

A real bug found during this build: the "spiky" pattern originally *randomized* which two of the four topics were high per student (matching the more general spec of "2 high, 2 low, randomize which 2"). That worked by luck at a small sample size (51 students), but at 200,000 rows the randomization averaged out — every dimension converged to ~0.5 across the whole "spiky" population, and the cluster centroid lost the unevenness signal entirely (KMeans found 2 near-identical mid-level clusters instead of 3 distinct archetypes, and the `assert len(set(labels)) == 3` check in the script failed). Fixed by using a **consistent** high/low topic split for the spiky pattern, which keeps the archetype statistically recoverable at any dataset size. See the comment in `train_cluster_model.py` for the full explanation.

Held-out results (20% test split, ground truth = the generating pattern, not the KMeans label):

Metric	Value
Test clustering accuracy	86.5%
Silhouette score (test sample)	0.31

Clustering accuracy is measured by: fit KMeans on the train split, map each resulting cluster to its majority true pattern, then check how often a held-out point's assigned cluster matches its actual generating pattern.

Learned centroids (order: Fractions, Ratios, Linear Equations, Percentages — **this exact order matters**, since `app/api/dashboard/route.ts` builds each student's mastery vector in this fixed order to match):

Label	Centroid
Needs Support	[0.26, 0.28, 0.26, 0.28]
On Track	[0.75, 0.77, 0.75, 0.77]
Uneven — targeted help needed	[0.77, 0.26, 0.78, 0.26]

TypeScript inference: Euclidean distance from the input vector to each of the 3 centroids; returns the nearest one's label.

Model 3 — Time-to-mastery estimation {#model-3}

File: `scripts/train_time_to_mastery_model.py` -> `lib/models/time-to-mastery-model.json` -> `lib/ml/time-to-mastery-model.ts`

Predicts: how many practice attempts a student needs to reach mastery (`score >= 0.8`) on a topic, given their current mastery and an "aptitude" estimate (their steady-state per-attempt success probability).

Not wired into the UI yet. Trained and unit-tested only.

Training data (`data/time_to_mastery_training_data.csv` , 200,000 rows, columns `initial_mastery` , `aptitude` , `attempts_to_mastery`): this is the one model whose target isn't a formula — it's generated by **actually simulating** the real `updateMastery()` recurrence from `lib/adaptive-engine.ts` (`K = 0.15`) forward, attempt by attempt, capped at 150 attempts (rows that never reach 0.8 within the cap are "censored" at 150). For each of 200,000 (`initial_mastery` , `aptitude`) pairs, **10 independent trials are simulated and averaged** — a single trial is extremely noisy (attempts-to-mastery is a near-geometric random variable), so averaging a handful of replicates per student gives a far more stable regression target, the same way you'd average repeated measurements of any noisy quantity.

Why quadratic features were necessary: a plain linear fit on the raw (`initial_mastery` , `aptitude`) pair capped out around $R^2 = 0.40$ no matter how much replicate-averaging reduced the noise — because the true relationship is genuinely curved (attempts blow up as aptitude approaches the 0.8 target from above) and a straight line architecturally can't capture that. Adding `mastery2` , `aptitude2` , `mastery x aptitude` as extra features (still a *linear regression*, just over an expanded feature set) fixed this:

```
// lib/ml/time-to-mastery-model.ts
const raw =
  w.mastery * mastery + w.aptitude * aptitude +
  w.masterySq * mastery**2 + w.aptitudeSq * aptitude**2 +
```

```
w.masteryAptitude * mastery * aptitude + bias;  
return Math.round(clamp(raw, 1, 150));  
}
```

Held-out results (20% test split, 40,000 rows):

Metric	Value
Test R ²	0.894
Test MAE	6.1 attempts

(For comparison: the naive 2-feature linear fit without replicate averaging or quadratic features scored $R^2 = 0.40$, MAE = 20.2 attempts — this is the one model in the set that genuinely needed feature engineering to hit the target range, not just a noise-level tuning knob.)

Model 4 — Class-wide topic priority recommender {#model-4}

File: `scripts/train_topic_priority_model.py` -> `lib/models/topic-priority-model.json` -> `lib/ml/topic-priority-model.ts`

Predicts: a 0-100 urgency score for "how much should the teacher focus whole-class instruction on this topic right now", from three class-wide stats: average mastery, fraction of students struggling (`mastery < 0.5`), and the spread (std dev) of mastery across the class.

Not wired into the UI yet. Trained and unit-tested only. Intended use: a "class focus" panel on the teacher dashboard ranking all 4 topics by urgency (`rankTopicsByPriority()` is already implemented and tested for this).

Training data (`data/topic_priority_training_data.csv` , 200,000 rows, columns `avg_mastery`, `pct_struggling`, `std_mastery`, `priority`): ground truth is `100 * sigmoid(6*(0.5 - avg_mastery) + 3*pct_struggling + 4*std_mastery)` plus Gaussian noise (std 4, tuned for the target R²).

Held-out results:

Metric	Value
Test R ²	0.859
Test MAE	10.2 priority points (0-100 scale)

Learned weights: `avg_mastery`: -86.1 , `pct_struggling`: +18.9 , `std_mastery`: +43.0 — signs match the design intent (low mastery, more struggling students, and more unevenness all raise priority).

Model 5 — At-risk / declining-trend detector {#model-5}

File: `scripts/train_at_risk_model.py` -> `lib/models/at-risk-model.json` -> `lib/ml/at-risk-model.ts`

Predicts: whether a student is "at risk" on a topic — distinct from the static weak-topic rule already in the engine (`mastery < 0.5`) because this one also factors in **recent trend** (is their accuracy improving or declining lately),

not just the current level. A student sitting at 0.55 mastery but trending sharply downward can be flagged before they cross the static threshold.

Not wired into the UI yet. Trained and unit-tested only.

Training data (`data/at_risk_training_data.csv` , 200,000 rows, columns `mastery`, `trend`, `attempts_count`, `at_risk`): ground truth is `sigmoid(1.5 * (-6*(mastery-0.4) - 10*trend))` , `trend` uniform on `[-0.3, 0.3]`. The `1.5` steepness multiplier is the noise-tuning knob.

Held-out results:

Metric	Value
Test accuracy	87.2%
Test ROC AUC	0.95
Bayes-optimal ceiling	87.2%

Again essentially at the theoretical ceiling.

estimateTrend() helper (`lib/ml/at-risk-model.ts`): since the app doesn't store a mastery time series (only discrete `Attempt` rows), trend is estimated from a sequence of recent correct/incorrect outcomes as (*accuracy of the newer half*) - (*accuracy of the older half*). Returns 0 (neutral) with fewer than 4 attempts — not enough signal either way.

Retraining

Every `train_*.py` script is self-contained: it regenerates its CSV in `data/` , retrains, re-evaluates on a fresh 80/20 split, and overwrites its JSON in `lib/models/` . All five use `np.random.default_rng(42)` , so reruns are **deterministic** — rerunning without changing any script produces byte-identical output.

```
source venv/bin/activate
python scripts/train_<name>_model.py
```

After retraining, re-run the matching TypeScript sanity test (`npx tsx scripts/test-<name>-model.ts`) to confirm the new weights still pass the sign/ordering checks before trusting them in the app.

API Reference

All routes are Next.js App Router route handlers under `app/api/`. There is no authentication — `studentId` is a plain `cuid()` the client holds in `localStorage` (see `lib/session.ts`) and passes explicitly.

POST /api/student

Creates a student and seeds a `Mastery` row (score `0.5`) for every topic.

File: `app/api/student/route.ts`

Request body:

```
{ "name": "Priya Sharma" }
```

Response 200 :

```
{ "student": { "id": "cuid...", "name": "Priya Sharma" } }
```

Response 400 : `{ "error": "Name is required" }` if `name` is empty/missing.

GET /api/student

Lists all students, alphabetically by name. (Not currently used by any page — available for future use.)

Response 200 :

```
{ "students": [{ "id": "...", "name": "..."}, ...] }
```

GET /api/student/[id]

One student's masteries, weak topics, and full attempt history — powers `/revision` and the dashboard's student drill-down panel.

File: `app/api/student/[id]/route.ts`

Response 200 :

```
{
  "student": { "id": "...", "name": "..."},
  "masteries": [
    { "topicId": "...", "topicName": "Fractions", "score": 0.63 },
    ...
  ],
  "weakTopics": [
    { "topicId": "...", "topicName": "Ratios", "score": 0.31, "attemptCount": 7 }
  ],
  "attempts": [
    {
      "id": "...",
```

```

    "questionText": "2/3 + 1/6 = ?",
    "topicName": "Fractions",
    "difficulty": 2,
    "correct": true,
    "createdAt": "2026-09-22T..."
  },
  ...
]
}

```

Attempts are ordered newest-first. `weakTopics` uses the same rule as the adaptive engine (`mastery < 0.5` AND `attemptCount >= 3`), sorted weakest first.

Response 404 : { "error": "Student not found" }

GET `/api/quiz/next?studentId=<id>&topicId=<optional>`

Returns the next question to serve, using the diagnostic-then-adaptive logic in `lib/quiz-service.ts`. Pass `topicId` (from the `/revision` page's "Practice this" button, or `?topic=` on `/quiz`) to restrict to one topic and skip the diagnostic phase entirely.

File: `app/api/quiz/next/route.ts`

Response 200 (diagnostic phase):

```

{
  "question": {
    "id": "...", "text": "2/3 + 1/6 = ?",
    "options": ["5/6", "3/9", "1/2", "4/6"],
    "difficulty": 2, "topicId": "...", "topicName": "Fractions"
  },
  "reason": "Diagnostic question 1 of 8 for Fractions – establishing your baseline.",
  "phase": "diagnostic",
  "diagnosticProgress": { "current": 1, "total": 8 }
}

```

Response 200 (adaptive phase): same shape, `phase: "adaptive"`, `diagnosticProgress` omitted, `reason` explains the topic choice (see [ADAPTIVE-ENGINE.md](#)).

Note: `options` never includes which index is correct — that's resolved server-side in `/api/quiz/answer` so it can never leak to the client.

Response 400 : { "error": "studentId is required" } **Response 404** : { "error": "No question available" } (e.g. restricted to a topic with no questions matching the target difficulty band, which shouldn't happen with the seeded data but is handled defensively)

POST `/api/quiz/answer`

Records an attempt, updates that topic's `Mastery` row, returns whether the answer was correct.

File: `app/api/quiz/answer/route.ts`

Request body:

```
{ "studentId": "...", "questionId": "...", "selectedIdx": 2 }
```

Response 200 :

```
{
  "correct": false,
  "correctIdx": 0,
  "topicId": "...",
  "mastery": 0.425
}
```

`mastery` is the topic's *new* score after this attempt's update.

Response 400 : missing/wrong-typed fields. **Response 404 :** { "error": "Question not found" }

GET /api/dashboard

Everything the teacher dashboard needs in one call: per-student mastery by topic, archetype badge, weak topics, and attempt counts.

File: `app/api/dashboard/route.ts`

Response 200 :

```
{
  "topics": [{ "id": "...", "name": "Fractions" }, ...],
  "students": [
    {
      "id": "...", "name": "Aisha Khan",
      "masteryByTopic": { "<topicId>": 0.78, ... },
      "totalAttempts": 25, "correctAttempts": 23,
      "weakTopics": [],
      "archetype": "On Track"
    },
    ...
  ]
}
```

`archetype` comes from Model 2 ([ML-MODELS.md](#)) — the mastery vector passed to it is built in a **fixed topic order** (Fractions, Ratios, Linear Equations, Percentages) regardless of the order `topics` is returned in, because that's the order the model was trained on.

Database

PostgreSQL via Prisma 6. (The build started on SQLite for a zero-setup first pass, then was migrated to Postgres so the data could be browsed in DBeaver like a normal relational database — see "Migration history" below.)

Schema

File: `prisma/schema.prisma`

```
model Student {
  id      String    @id @default(cuid())
  name    String
  attempts Attempt[]
  mastery Mastery[]
}

model Topic {
  id      String    @id @default(cuid())
  name    String
  questions Question[]
  mastery Mastery[]
}

model Question {
  id      String    @id @default(cuid())
  topicId String
  topic   Topic     @relation(fields: [topicId], references: [id])
  text    String
  options String      // JSON-stringified array of 4 option strings
  answerIdx Int        // index into `options` of the correct answer
  difficulty Int       // 1, 2, or 3
  attempts Attempt[]
}

model Attempt {
  id      String    @id @default(cuid())
  studentId String
  student Student   @relation(fields: [studentId], references: [id])
  questionId String
  question Question @relation(fields: [questionId], references: [id])
  correct Boolean
  createdAt DateTime @default(now())
}

model Mastery {
  id      String    @id @default(cuid())
  studentId String
  student Student   @relation(fields: [studentId], references: [id])
  topicId String
  topic   Topic     @relation(fields: [topicId], references: [id])
  score   Float      @default(0.5) // 0-1, Elo-style running estimate

  @@unique([studentId, topicId]) // one Mastery row per (student, topic) pair
}
```

`options` is stored as a JSON string (not a native array) because SQLite had no array column type when the schema was first written; kept as-is after the Postgres migration for consistency —

`JSON.parse()` / `JSON.stringify()` at the application boundary in `lib/quiz-service.ts` and `prisma/seed.ts`.

`Mastery` has exactly one row per `(studentId, topicId)` pair, enforced by the `@@unique` constraint — `lib/adaptive-engine.ts`'s `updateMastery()` result is written via `prisma.mastery.upsert()` against that same key.

Local setup (Postgres via Homebrew)

```
brew install postgresql@18
brew services start postgresql@18
createdb erodehack
```

`.env` :

```
DATABASE_URL="postgresql://<your-macos-username>@localhost:5432/erodehack"
```

Local Homebrew Postgres uses trust auth for the local Unix user by default, so no password is needed in the connection string.

```
npx prisma migrate dev # applies prisma/migrations/ to the erodehack database
npm run seed           # runs prisma/seed.ts — loads 4 topics x 10 questions
```

Connecting with DBeaver

Database -> New Database Connection -> PostgreSQL, then:

Field	Value
Host	<code>localhost</code>
Port	<code>5432</code>
Database	<code>erodehack</code>
Username	your macOS username
Password	<i>(leave blank)</i>

Test Connection -> Finish. You'll see `Student`, `Topic`, `Question`, `Attempt`, `Mastery` under the `public` schema.

Migrations

Directory: `prisma/migrations/` — one migration so far (`..._init`), generated by `npx prisma migrate dev --name init` against the Postgres datasource. Check current status any time with:

```
npx prisma migrate status
```

Migration history: SQLite -> Postgres

The project was originally scaffolded with `datasource { provider = "sqlite" }` and a `prisma/dev.db` file — genuinely zero-setup, useful for the first build pass. When Postgres/DBeaver access was needed:

1. Changed `provider = "sqlite"` -> `provider = "postgresql"` in `schema.prisma`
2. Updated `DATABASE_URL` in `.env` to a Postgres connection string
3. Deleted the old SQLite-flavored `prisma/migrations/` (its SQL dialect doesn't carry over) and regenerated with `prisma migrate dev --name init` against the new datasource
4. Re-ran `npm run seed` and the demo-data simulation script

No application code changed — `lib/prisma.ts`'s `PrismaClient` and every query in `app/api/**` are database-engine-agnostic through Prisma.

Seeding

`prisma/seed.ts` creates the 4 fixed topics (Fractions, Ratios, Linear Equations, Percentages) and 10 questions each (3-4 per difficulty level, 1-3), with plausible (not obviously wrong) distractor options. Re-running it appends a second copy of everything — it doesn't check for existing data, so wipe the tables first if you need a clean reseed:

```
psql erodehack -c 'TRUNCATE "Attempt", "Mastery", "Question", "Topic", "Student" CASCADE;'
npm run seed
```